# CSC612M Midterm Milestone

## Alzheimer's Disease Drug Combination Search: Benchmarking SIMD, CUDA, and QAQA

Campo, Macayan, Madridijo

November 9, 2026

**Abstract**

Alzheimer's disease (AD) involves many biological processes at once, so treatment is expected to need combinations of drugs, often ones already approved for other conditions. Computationally, choosing $k$ drugs from $n$ candidates is a search over $\binom{n}{k}$ combinations. We follow a network-medicine rule for good combinations: each drug's protein targets should lie close to the AD disease proteins, while the drugs' target sets should stay separated from one another. We encode this rule as a single integer-valued score ("energy") over every combination, in a quadratic form suited to both annealing and QAOA, and search for its minimum in three ways: (1) An exhaustive GPU search checks every combination and computes the true minimum, which serves as ground truth. (2) A simulated-annealing search is implemented as sequential C++, as an AVX2SIMD kernel written in assembly, and as a CUDA kernel. All three implementations use the same per-chain random streams and fixed-point integer arithmetic, ideally returning identical solutions. Any timing difference therefore reflects the underlying hardware rather than the algorithm. We report throughput and speedup over the sequential baseline as $n$, $k$, and the number of parallel chains grow. (3) Moving away from traditional computing, we further implement a Quantum Approximate Optimization Algorithm (QAOA) running on a simulated quantum computer implemented via NVIDIA cuQuantum. In benchmarking, we measure how close it gets to the exact minimum, along with its simulation time and memory, which grow exponentially with $n$. As a limitation, we show that the results are for computational hypotheses for later experimental testing, not treatment recommendations, and because the quantum computer is simulated on classical hardware, we strictly cannot conclude quantum speedup.

# 1 Introduction

## 1.1 The Case for Combination Search in Alzheimer's Disease

Alzheimer's disease (AD) is the most common cause of dementia. Its two hallmarks are plaques of amyloid-$\beta$ outside nerve cells and tangles of tau protein inside them, and in lesser cases: inflammation, synaptic loss, vascular damage and altered brain metabolism. Drugs aimed at a single target are shown to be ineffective against AD, and the approved anti-amyloid antibodies only modestly slow decline, so previous work suggests that several processes may need to be treated at once. The annual review of the AD drug-development pipeline reports that 35% of the drugs now in AD clinical trials are *repurposed*, meaning they are already approved for another indication, and that these drugs make up almost half of Phase 2 agents [1]. Repurposed drugs give us a pool of approved candidates with known safety profiles, which makes a search over their combinations practical. Clinical trials of combinations are slow and expensive, because the number of trial arms grows with every drug added,

so a computational method that prioritizes a short list of promising combinations can notably aid in generating hypotheses prior to these programs.

### 1.2 A Network Rule for Drug Combinations

We based our proposal under the framework of network medicine treating the cell's proteins as a graph, the human *interactome*. Proteins linked to the same disease cluster together in this graph, forming a *disease module* [2, 3]. A drug acts on a few proteins, its *targets*. Drugs whose targets are unusually close to the disease module are more likely to treat that disease [4].

For combinations, Cheng, Kovács and Barabási [5] looked at how the targets of two drugs relate to each other and to the disease module. Of the six possible patterns, only one went with combinations that actually work in patients: both drugs are close to the disease module, but their targets are in different parts of the network. They dub this observation as *complementary exposure*.

### 1.3 Motivation as to how this is a Computational Problem

Choosing $k$ drugs out of $n$ means searching $\binom{n}{k}$ combinations. This is trivial for $n = 20$, $k = 3$ (1,140 combinations), but it grows quickly: $\binom{100}{5} \approx 7.5 \times 10^7$, $\binom{1000}{5} \approx 8 \times 10^{12}$, and $\binom{1000}{10} \approx 2.6 \times 10^{23}$. We observe that the search consists of many independent, regular operations, and each of our three methods stresses a different part of the machine:

- **Exact search** scores every combination, and each score is independent of the others, so one GPU thread can check a slice of them. We expect it to be limited by decoding combination indices and reading the score table.
- **Simulated annealing** runs many independent search chains. Every accepted step inside a chain adds one row of the score table to a length-$n$ array and subtracts another, which is what a `ymm` register (eight 32-bit integers) and `vpaddd`/`vpsubd` are for. Only accepted steps do this work, so we expect the speedup to depend on the acceptance rate, which tends to fall as the temperature drops.
- **Quantum simulation** stores $2^n$ complex amplitudes and, for every gate, applies the same small matrix to every group of two or four of them. Each gate streams the whole state through memory, so we expect it to be limited by memory bandwidth, and memory capacity caps $n$ at about 30 qubits on our single GPU.

SIMD instructions (AVX2) and GPUs (CUDA) suit this kind of regular, data-parallel work, but not every part speeds up equally. Part of our contribution is measuring where the speedup comes from and where it stops, and comparing all three methods in the range of $n$ where all of them can run.

### 1.4 The Quantum-based Addition

Extending beyond traditional computing. our scoring function has single terms and pair terms only, which is the shape of a Quadratic Unconstrained Binary Optimization (QUBO) problem [6, 7]. QUBOs are the standard input for quantum optimization algorithms such as QAOA [8]. We have no quantum computer, but NVIDIA cuQuantum [9] simulates one with an NVIDIA GPU that we are able to access. Because our instances are small enough to solve exactly, we can measure precisely how close simulated QAOA gets to the true best answer. A recent study suggests network-based drug-combination prediction as a QUBO and solved it with simulated quantum annealing for four other diseases [10]; we found no study that applies QAOA to AD drug combinations.

### 1.5 Expected Deliverables

1. A documented and checked scoring function (QUBO) for AD drug combinations, built from publicly-accessible drug data.
2. An exact GPU search over all valid combinations, which gives the reference answer.
3. One simulated-annealing algorithm written in three approaches (sequential scalar C++, AVX2 assembly, CUDA), with identical results, and an execution-time comparison.
4. QAOA, simulated with CUDA-Q and cuStateVec, measured against the exact answer.

.

### 1.6 Relationship to Sister Work: Network-Screening project

This project replaces the optional Module 5 of the sister project and can run independently. It needs two things for each candidate drug: its closeness score $z$ and its list of targets. It can take these from the sister project's output, or compute them for a hand-picked candidate list with a small script of its own (Decision D1, Section 8). The two projects agree on one table format (**??**), so either source can be plugged in without changing the underlying solvers.

## 2 Review of Related Literature

### 2.1 Network Proximity and the Disease Module

Network medicine studies disease through the network of molecular interactions [2]. Menche et al. [3] showed that disease proteins form connected modules in the incomplete human interactome and defined the *network separation* $s_{AB}$ of two protein sets, which is negative when the sets overlap in the network and positive when they are apart. Guney et al. [4] turned closest-distance proximity into a drug screen, standardizing it against degree-matched random target sets to obtain a $z$-score; drugs with significantly negative $z$ are enriched for known treatments. Cheng et al. [11] applied the approach to cardiovascular disease with population-based validation.

### 2.2 Network-based Drug Combinations

Cheng, Kovács and Barabási [5] combined the two measures: for a drug pair $(A, B)$ and disease module $S$, they considered the proximity of each drug to $S$ and the separation $s_{AB}$ between the drugs' targets. Using approved combinations for hypertension and cancer, they identified that only the *complementary exposure* pattern (both drugs close to $S$, targets separated, $s_{AB} \geq 0$) correlated with therapeutic effect, and that the drug–drug proximity $z$-score by itself could not distinguish approved combinations from random pairs. This finding is our objectives' scientific basis: reward proximity of each drug to the AD module and penalize overlap between the selected drugs' targets.

### 2.3 Network Proximity in Alzheimer's Disease

Fang et al. [12] modeled AD endophenotype modules, screened drugs by proximity, and used insurance-claims data to associate sildenafil use with lower AD incidence. Later pipelines ranked drugs for different pathological stages of AD [13] and combined proximity ranking with heart-rhythm (Long QT) risk screening [14]. The AD genes used to build modules come from large genome-wide association studies [15], and the annual pipeline reviews [16, 1] list drugs currently in AD trials,

including repurposed agents. These studies rank single drugs; we found none that optimizes AD combinations with the complementary-exposure principle.

### 2.4 QUBO Formulations and Constraints

Many selection problems can be written as QUBOs, $\min_{\mathbf{x}\in\{0,1\}^n} \mathbf{x}^\top Q\mathbf{x}$ [6, 7]. Constraints such as "exactly $k$ items" are usually added as quadratic penalties, $A(\sum_i x_i - k)^2$. This is convenient but has known costs: the penalty couples every pair of variables, making $Q$ dense, and a penalty weight that is too small allows invalid solutions while one that is too large flattens the energy landscape that heuristics and quantum algorithms rely on [6].

### 2.5 Parallel simulated annealing

Simulated annealing (SA) [17] is the standard classical heuristic for QUBO and Ising problems. A single SA chain is inherently sequential, since every step depends on the previous state, so parallel implementations usually run many independent chains (replicas) or exchange states between chains at different temperatures (parallel tempering) [18]. Isakov et al. [19] showed how careful data layout, precomputed acceptance tables and multi-spin coding make SA on Ising models much faster on CPUs. Counter-based random-number generators such as Philox [20] produce the same random stream on CPU and GPU from a (key, counter) pair, which makes cross-platform reproducibility possible.

### 2.6 QAOA and Constraint-preserving Mixers

QAOA [8] alternates a cost layer $e^{-i\gamma H_C}$ with a mixing layer $e^{-i\beta H_M}$, starting from the uniform superposition, and tunes the $2p$ angles classically. Hadfield et al. [21] generalized it to the Quantum Alternating Operator Ansatz, in which the mixer and initial state are chosen so the state never leaves the set of valid solutions. For "exactly $k$ ones" constraints, *XY mixers* preserve the number of ones (the Hamming weight) [22], and a natural initial state is the Dicke state, the equal superposition of all weight-$k$ strings, which can be prepared deterministically with $O(kn)$ gates [23]. Cook, Eidenbenz and Bärtschi [24] studied this ansatz on Maximum $k$-Vertex Cover, a problem with the same cardinality structure as ours, comparing ring and complete-graph XY mixers. Zhou et al. [25] described parameter-initialization heuristics that make QAOA training at larger depth $p$ practical.

### 2.7 Quantum Approaches to Drug Combinations

Ramos, Coutinho and Magano [10] built a QUBO from the complementary-exposure principle and sampled it with simulated quantum annealing, testing it on diabetes mellitus, rheumatoid arthritis, asthma and brain neoplasms against experimentally validated combinations. Their interactome and drug–target data come from Cheng et al. [5], and their code is public. They did not study AD, gate-model QAOA, constraint-preserving mixers, or parallel classical solvers.

### 2.8 GPU Simulation of Quantum Circuits

NVIDIA cuQuantum [9] features cuStateVec (state-vector simulation) and cuTensorNet (tensor-network simulation). It is a simulation library, not a quantum processor or an optimizer. CUDA-Q [26] is NVIDIA's quantum programming framework; its default `nvidia` target runs single-GPU state-vector simulation through cuStateVec, with an `fp64` option for double precision [26]. CuStateVec's

documented requirements include compute capability 7.0 or higher (Volta, Turing, Ampere, Ada, Hopper) [27] which we have access to.

### 2.9 Synthesis and Research Gap

| Work | What it does | AD | Comb. | Parallel classical | Gate QAOA |
|---|---|---|---|---|---|
| Guney et al. [4] | Proximity $z$-score drug screen | – | – | – | – |
| Cheng et al. [5] | Complementary exposure for combinations | – | ✓ | – | – |
| AD proximity studies [12, 13, 14] | AD single-drug repurposing | ✓ | – | – | – |
| Isakov et al. [19] | Optimized CPU SA for Ising | – | – | ✓ | – |
| Cook et al. [24] | XY-mixer QAOA, cardinality constraint | – | – | – | ✓ |
| Ramos et al. [10] | Combination QUBO, simulated quantum annealing | – | ✓ | – | – |
| Sister project | GPU proximity screen for AD | ✓ | – | ✓ | – |
| **This work** | AD combination QUBO; SA on scalar, AVX2 and CUDA; exact GPU reference; QAOA vs. exact | ✓ | ✓ | ✓ | ✓ |

Table 1: How this project relates to prior work

Prior work work and exists distinct from AD solutions or computational modeling: a validated network principle for combinations, AD-specific proximity studies, fast parallel annealing, constraint-preserving QAOA, and one quantum-annealing study of combination QUBOs outside AD. What remains to be missing is (a) a carefully validated AD combination QUBO, (b) a controlled comparison of one combination-search algorithm across scalar, SIMD and SIMT hardware, and (c) a measurement of gate-model QAOA against exact optima on these biology-derived instances.

## 3 Problem Statement

### 3.1 Problem

Network proximity can shortlist AD drug candidates, and complementary exposure computes what a good pair looks like. In turning this into a recommended *combination* of $k$ drugs, we define a search problem whose size grows as $\binom{n}{k}$. Three questions are therefore open: how to turn closeness, overlap and combination size into one score without inventing biological numbers; how much one search algorithm gains from SIMD and from a GPU, and why; and how well a quantum algorithm (QAOA), simulated on a GPU, finds the best combination compared with the exact answer.

### 3.2 Proposed Solution

We build the system as five modules. Each module has fixed inputs and outputs, so we can build and test them separately (Table 2). Modules 1–4 are the project core; Module 5 is the research extension and its novelty.

| Module | Input | Output | Runs on |
|---|---|---|---|
| 1. Candidate preparation | Interactome, drug–target list, AD genes, candidate list; *or* the sister project's table | Candidate table (targets, $z_i$) and a shortlist of $n$ drugs | CPU (Python) |
| 2. Problem construction | Shortlist, network distances | Rewards $w_i$, penalties $P_{ij}$, QUBO and Ising files | CPU (Python) |
| 3. Exact search | $w$, $P$, $k$ | The best combination, how many combinations tie for best | CPU and GPU |
| 4. Simulated annealing | $w$, $P$, $k$, settings | Best combination per run, run statistics, timings | CPU scalar, AVX2, GPU |
| 5. QAOA (extension) | Ising file | Sampled combinations and their scores | GPU (CUDA-Q + cuStateVec) |

Table 2: Pipeline modules. Every solver reads the same problem file and writes the same result format (**??**).

### 3.3 Research questions and hypotheses

**RQ1 (main): How much do SIMD and the GPU help?**

How do sequential C++, AVX2 and CUDA versions of the *same* simulated-annealing algorithm compare in speed, as the number of candidates $n$ grows? We measure the time for one search chain, the number of steps per second when many chains run at once, and the time until the exact best answer is found.

*Hypothesis:* The GPU wins once enough independent chains run to keep it busy (hundreds or more). AVX2 wins for a single chain. Inside a chain, the speedup is limited because only *accepted* steps do the vectorized array update, and late in the search most steps are rejected.

**RQ2 (main): Is the scoring function right?**

Does our QUBO always pick a valid combination of exactly $k$ drugs, equal to the best combination found by exact search, and how does the overlap dial $\lambda$ change which drugs are chosen?

*Hypothesis:* With the penalty weight set above the bound we prove in **??**, the QUBO and exact answers always agree. As $\lambda$ grows, the chosen combinations move from "the $k$ drugs closest to AD" toward complementary combinations.

**RQ3 (extension): How good is QAOA?**

On instances small enough to solve exactly, how often does simulated QAOA return the best combination, how many of its answers are valid $k$-drug combinations, and how does this change with circuit depth $p$ and problem size $n$?

*Hypothesis:* Most of QAOA's answers will be invalid, because only a tiny share of all yes/no patterns have exactly $k$ ones ($\binom{n}{k}/2^n$; 0.1% at $n = 20$, $k = 3$). Quality will improve with $p$ but remain well below exact search.

**RQ4 (supporting): Where does the time go?**

How is total run time split between data preparation, problem construction, solving and checking? Answered from timing logs at no extra cost.

RQ1 and RQ2 are the course core and do not depend on the quantum part. RQ3 is the main research question for a possible conference paper.

### 3.4 Objectives

**General objective.** To build a checked scoring function for AD drug combinations and compare exact, SIMD, GPU and simulated-quantum ways of finding the best combination.

**Specific objectives.**

1. Build a reproducible candidate table and shortlist from public data, with clear documentation where every input comes from.
2. Build the QUBO and Ising files and prove, then test, that they return the right answer.
3. Implement exact search on CPU and GPU and confirm they agree.
4. Implement one simulated-annealing algorithm in sequential C++, AVX2 assembly and CUDA; show that all three make identical decisions; compare their execution times.
5. Run QAOA in CUDA-Q on the simulator and measure its answers against the exact search.

### 3.5 Scope and limitations

- **Public data only.** No patient data.
- **Hypotheses, not recommendations.** A high-scoring combination is a computer-generated idea. It is not evidence that the combination is effective, synergistic or safe.
- **Access to a upper mid-range GPU.** A GeForce RTX 5070Ti with 16 GB of GDDR7 VRAM. This limits the quantum simulation to about 30 qubits (one qubit per drug). It is sufficient for the classical solvers at our sizes.
- **We cannot claim any quantum speedup.** The quantum computer is simulated, and at these sizes exact search is fast. The quantum part measures answer quality only.
- **Weak biological validation.** Very few AD drug combinations have trial results, so we check whether chosen combinations follow the complementary-exposure pattern, not whether they work.
- **Cut for time.** Safety and side-effect data, splitting large problems into QAOA-sized pieces, a constraint-preserving QAOA variant, and several extra performance experiments were left out. **??** lists each one, why it was cut, and how it may be added in the future.

### 3.6 Significance

The proposed study observes one algorithm on three kinds of hardware and explains where data-level parallelism helps and where a step-by-step dependency may limit it. Furthermore, it gives a realistic, biology-based test problem for quantum optimization, with exact answers to compare against, and the first QAOA study of AD drug combinations that we are aware of.

## 4 Methodology

### 4.1 Description of the Proposed Project

#### 4.1.1 Inputs

All inputs are publicly accessible. We record the versions and checksum so our results can be reproduced.

- **Human interactome.** The protein–protein interaction graph compiled by Cheng et al. [5], restricted to its largest connected piece. Using the same graph as [10] makes our results comparable with theirs. If both projects run, the sister project's BioGRID graph [28] can be used instead.

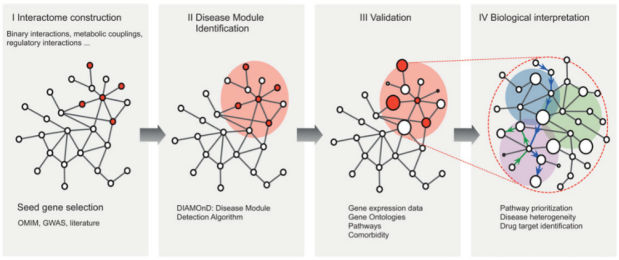



Figure 1: Example of identifying a network module in the Human interactome.

- **Drug–target list.** The drug–target table of Cheng et al. [5], filled in from ChEMBL [29] or DGIdb [30] for any candidate it lacks. Only targets present in the interactome are kept.
- **AD disease module.** The same definition as the sister project: genes from the largest recent AD genetic study [15] plus well-known AD genes (e.g., *APP*, *PSEN1*, *PSEN2*, *APOE*, *MAPT*).
- **Candidate list** (Decision D1). Either the sister project's ranked output, or a hand-picked ("curated") list: repurposed drugs from the 2025 and 2026 AD pipeline reviews [16, 1], the four approved symptomatic AD drugs (donepezil, rivastigmine, galantamine, memantine), and drugs suggested by published AD network studies [12, 13, 14]. Antibody drugs whose targets are not in the interactome are left out.
- **Scaling pool.** For speed experiments only: every drug in the drug–target table with at least one target in the interactome (expected: roughly a thousand).
- **Settings.** Combination size $k \in \{2, 3, 4\}$ for biological results (up to 8 for speed tests); overlap weight $\lambda$; number of SA chains and steps; random seeds.

We note that the solvers does not consider genuine biology principles or theories. Instead, they see a list of $n$ numbers (how close each drug is to AD), an $n \times n$ table (how much each pair of drugs overlaps), and the number $k$. Everything biological is simply a label attached to those numbers.

More precisely, the pipeline answers one question: *which $k$ drugs together cover the AD module best without overlapping each other?*

1. **Score each drug.** Take drug $i$'s closeness score $z_i$ (**??**). Only drugs with $z_i < 0$ (closer than chance) are kept. The reward is $w_i = -z_i$, capped and scaled to the range 0–1.
2. **Score each pair.** Compute the separation $s_{ij}$ between the targets of drugs $i$ and $j$ [3]. Overlapping pairs ($s_{ij} < 0$) get a penalty $P_{ij} = -s_{ij}$, scaled to 0–1; all other pairs get 0.
3. **Find the best team.** Find the set $\mathcal{S}$ of exactly $k$ drugs with the lowest energy

$$E(\mathcal{S}) = -\sum_{i \in \mathcal{S}} w_i + \lambda \sum_{\{i,j\} \subseteq \mathcal{S}} P_{ij}. \tag{1}$$

The first sum rewards closeness to AD; the second punishes overlap. Low energy therefore means complementary exposure [5].

For three drugs A, B, C the energy is: minus (reward of A + reward of B + reward of C), plus $\lambda$ times (overlap of A–B + overlap of A–C + overlap of B–C). Pick the three drugs that make this number as small as possible.

**(i) Processes parallelized with SIMD (AVX2) and SIMT (CUDA).**

1. **Exact search over all combinations**. The $\binom{n}{k}$ combinations are numbered in order and split into equal slices, one per GPU thread. Each thread finds its first combination from its slice number, walks through its slice, and remembers its best. The threads then combine their bests with a minimum reduction. A sequential C++ version gives the cross-check and the timing baseline.
2. **Simulated annealing**. Each chain holds exactly $k$ drugs and repeatedly tries swapping one chosen drug for one unchosen drug. A helper array $g$ of length $n$ makes the score change of any swap an $O(1)$ calculation (**??**). When a swap is accepted, $g$ is updated with $g \leftarrow g + \lambda(P_{:,j} - P_{:,i})$, an array add and subtract over $n$ elements.
   - **AVX2**: this update is a hand-written assembly loop using `ymm` registers, eight 32-bit integers per instruction.
   - **CUDA**: thousands of chains run at once. Version 1 uses one thread per chain.
3. **QAOA simulation** (Module 5). The cuStateVec library applies each quantum gate to all $2^n$ numbers of the simulated state in parallel on the GPU. We do not write this kernel, but we time it and measure its memory use.

**(ii) Related Works and How We Differ.** We acknowledge that the complementary exposure [5] and its quantum-annealing QUBO [10] inspires the scoring idea. Optimized simulated annealing [19] and QAOA [8] suggests our solver framework. Our implementation differs in that it:

- targets AD, and proves and tests that the QUBO gives the right answer instead of tuning its settings by trial and error;
- searches only valid combinations in the classical solvers, so no penalty setting is needed there;
- writes *one* SA algorithm for scalar, SIMD and GPU hardware with shared random numbers and integer arithmetic, so the three versions make identical decisions and their run times can be compared fairly;
- measures QAOA against the exact best answer on every instance, instead of reporting quantum results alone.

**(iii) Parallelization plan.** Data moves to the GPU once and results come back only once.

**(iv) Processes not parallelized.**

- Data Preprocessing and ETL: Downloading data, matching gene and drug identifiers, building the interactome and target lists.
- Computing closeness scores for the curated list when the sister project's output is not used (a small Python script; not part of the contribution).
- Shortest-path distances among target proteins and the separation table $s_{ij}$ (breadth-first search from a few thousand proteins at most; fast enough on the CPU).

| Step | How the work is split | Limited by |
|---|---|---|
| Exact search (GPU) | Combinations numbered $0, \ldots, \binom{n}{k} - 1$ and split into equal slices, one per thread. Each thread decodes its first combination, steps to the next one in order, and re-scores only the drugs that changed. Warp and block minimum reduction of (energy, combination number, tie count). | Arithmetic |
| SA, AVX2 | Per chain: drawing the random swap and computing its score change is scalar code. Accepted swaps update $g$ with `vmovdqa`/`vpaddd`/`vpsubd` over eight lanes, unrolled four times. | Share of steps accepted; memory bandwidth for large $n$ |
| SA, CUDA | One thread per chain; $g$ in the thread's local memory. | Uncoalesced memory access |
| QAOA | cuStateVec over $2^n$ numbers; the angle-tuning loop runs on the CPU. | GPU memory |

Table 3: Parallelization plan. Pseudocode and memory layouts are in **??**.

- Building the QUBO and Ising files and the annealing temperature schedule.
- Building QAOA circuits and running the angle-tuning optimizer (COBYLA), which calls the GPU simulator.
- Result analysis and plotting.

#### 4.1.2 Outputs

1. **Problem files**: candidate table, separation and penalty tables, QUBO and Ising files, each with a record of where its numbers came from.
2. **Best AD combinations** for each $k$ and $\lambda$, with the exact best answer, how many combinations tie for best, and the runners-up.
3. **Execution-time comparison**: sequential C++ vs. AVX2 vs. CUDA for SA, and CPU vs. GPU for exact search, across $n$ and $k$, with speedup charts.
4. **Correctness report**: CPU and GPU exact search agree; all SA versions make identical decisions; QUBO and Ising scores agree for every yes/no pattern on small problems.
5. **QAOA results**: how often QAOA finds the best combination, the share of valid answers, and simulator time and memory, for each depth $p$ and size $n$.

## 5 Proposed Target Implementation Platform

The RTX 5070 Ti is a consumer gaming card. That is enough for every classical part of this project, because the problems are small in memory (a 2,000-drug penalty table is 16 MB, which fits in the GPU's 48 MB L2 cache and in the CPU's 30 MB L3 cache). It is the limiting factor for the quantum simulation: simulating $n$ qubits needs $2^n$ complex numbers, so each extra qubit doubles the memory. In single precision a 30-qubit state takes 8 GB, so 16 GB runs out at roughly 30 qubits (about 29 in double precision).

# 6 Scope and Limitations

## 6.1 D1: Where the candidate drugs come from; Executed: B

| Option | Doability | Novelty |
|---|---|---|
| A. Sister project's ranked output | Depends on the other team (their list is due in Week 10). | Highest: a full screen-then-combine pipeline. |
| B. Curated list (pipeline reviews + approved AD drugs + published AD network hits) | High: tens of drugs; their closeness scores need only a small script. The data of [5] is ready to use. | Moderate: AD-specific and easy to explain, but small. |
| C. Both, through one shared table format | High: never blocked. | Like A when A arrives; like B otherwise. |

**Decision D1 (Executed: B)**

We built and tested everything on the curated list. We score the scaling pool with our own script and label those instances as speed tests, not biological results.

The curated list is small, which is good for biology and for exact answers, but too small to show interesting speedups. So the project uses two families of problems: small biological ones (about 12–24 drugs) for correctness and QAOA, and large ones (hundreds to a couple of thousand drugs) for speed tests.

## 6.2 D2: How to enforce "exactly $k$ drugs"

A commonly refenced textbook trick is to add a penalty $A(\sum_i x_i - k)^2$ to the energy. It works, but it links every pair of drugs, it needs a good value of $A$, and it makes almost every yes/no pattern invalid: for $n = 20$, $k = 3$, only 1,140 of about a million patterns (0.1%) have exactly three drugs [ESTABLISHED] (arithmetic).

**Decision Executed as-is**

- **Classical solvers** only ever look at valid combinations: exact search lists only $k$-drug combinations, and SA only swaps drugs in and out, so the count stays at $k$. No penalty is needed, and every answer is valid [PROPOSED].
- **QAOA** uses the penalty version, because standard QAOA cannot enforce a count by itself.

## 6.3 D3: Which search algorithm to parallelize

**Decision D3 (Executed: simulated annealing with swap moves)**

It is short to write, easy to explain, and has a clear vectorizable step. Running many independent chains is natural on a GPU. Other methods (tabu search, parallel tempering) were considered and left out.

### 6.4 D4: SIMD style

**Decision D4 (Executed: AVX2 assembly)**

SIMD has many variations of instruction sets such as AVX/AVX2/AVX-512 assembly, and the whole team has written `xmm`/`ymm` assembly before. The inner loop is written in NASM and called from C++. A C++ intrinsics version of the same loop is kept as a cross-check. AVX-512 is not used because many consumer CPUs lack it. For fairness, the sequential baseline is compiled with automatic vectorization turned off, and an automatically vectorized build is also reported.

### 6.5 D5: Making the three versions give identical answers

With floating-point numbers, the CPU and GPU round slightly differently, so after a few thousand steps their searches drift apart, and it becomes hard to tell a bug from rounding.

**Decision D5 (Executed: integer "deterministic mode")**

Scale all rewards and penalties to 32-bit integers, draw all random numbers from Philox with the same seed, and decide whether to accept a step with a shared lookup table. Then the scalar, AVX2 and CUDA versions must make exactly the same decisions, and comparing them catches bugs immediately. This is mainly a *debugging tool*: it makes the GPU version much easier to get right [PROPOSED].

### 6.6 D6: Quantum method

**Decision D6 (Incomplete: standard QAOA with the penalty)**

QAOA is the standard gate-based quantum method for this kind of problem and runs on CUDA-Q's GPU simulator. A variant that never produces invalid combinations (an "XY mixer" starting from a "Dicke state" [21, 22, 23]) would likely work better but needs a much harder circuit; it is a **stretch goal**, attempted only if standard QAOA is finished by Week 12 (**??**).

### 6.7 D7: Who does what

| Member | Responsibility |
|---|---|
| Member 1 (lead) | Data preparation; separation table; QUBO and Ising files and their tests; QAOA in CUDA-Q; C++ exact search |
| Member 2 | CUDA: exact-search kernel; SA kernels v1 (thread per chain) and v2 (warp per chain); the scalar C++ SA (the reference everyone checks against) |
| Member 3 | AVX2: assembly update loop and intrinsics twin; timing harness; benchmark scripts; plots and tables. Support Mmeber 2 |

Table 5: Roles. Member 1 only need to know QUBO or quantum computing: their solvers read only $w$, $P$ and $k$. Everyone reviews everyone's code, and Members 2 and 3 pair once on the CUDA SA kernel so knowledge is not held by one person.

### 6.8 Priorities

| Priority | Features |
|---|---|
| **MUST** | [DONE] Frozen data and candidate table; separation table; QUBO/Ising files with tests; exact search on CPU and GPU; SA in scalar C++, AVX2 assembly; timing tables. |
| **SHOULD** | [TODO] CUDA (v1) with deterministic mode; CUDA SA v2 (warp per chain); large-instance speed tests; standard QAOA on small instances; effect of $\lambda$ on the chosen drugs. |
| **OPTIONAL** | XY-mixer QAOA with Dicke state (stretch goal). |
| **OUT** | Everything in **??**: safety and side-effect data, splitting large problems for QAOA, parallel tempering, multi-core and other extra experiments, real quantum hardware. |

Table 6: Feature priorities.

### 6.9 Novelty Arguement

The individual methods are established: complementary exposure [5], simulated annealing [17], QAOA [8] and GPU state-vector simulation [9]. What is new is putting them together and measuring them carefully: (1) an AD-specific combination QUBO with a proven penalty setting and exhaustive tests; (2) one combination-search algorithm on scalar, SIMD and GPU hardware with identical decisions, so speed differences come from the hardware alone; and (3) to our knowledge, the first QAOA study of AD drug combinations, measured against exact answers. For a student research conference, (2) and (3) are the main claims.

## 7 Mid-Project Milestone Update

**Milestone 2 (Week 10) deliverable.** We present a live demo on the curated AD drugs attached in Github that shows:

1. the checked scoring function: on a 16-drug instance, an exhaustive Python check over all $2^{16}$ yes/no patterns confirms that the QUBO's best pattern is a valid $k$-drug combination and equals the exact-search answer;
2. CPU and GPU exact search returning the same best combination and the same tie count;
3. SA in scalar C++, AVX2 assembly;
4. a first sequential-vs-parallel timing table for exact search and SA;
5. the first best AD combinations for $k \in \{2, 3\}$ at two values of $\lambda$.

**Schedule.** Table 10 splits the work by member (roles in Table 8). Week 7 includes a deliberate CUDA ramp-up for Member 2 (and Member 3, who reviews the CUDA code).

| Week | Member 1 (lead) | Member 2 (CUDA) | Member 3 (AVX2, benchmarks) |
|---|---|---|---|
| 6 | Proposal (Milestone 1). Download and freeze data; fix table format. | Install CUDA toolkit; run the samples on the GTX 1660. | Set up repository, CMake, NASM build; Philox header. |
| 7 | Candidate table, closeness scores, separation table; QUBO/Ising builder; Python exhaustive checker. | **CUDA ramp-up**: vector add; a min-reduction kernel; a kernel that gives each thread its own Philox stream. | C++ exact search; reference energy function ("the judge"); timing harness. |
| 8 | Scalar C++ SA with integer deterministic mode (the reference). | GPU exact-search kernel; compare with Member 3's C++ version. | AVX2 assembly update loop and intrinsics twin; unit-test against scalar loop. |
| 9 | Formulation tests T1–T5 (**??**); first AD results. | CUDA SA v1 (thread per chain); cross-platform test T7. | AVX2 SA (scalar loop + assembly update); test T7; first timing table. |
| 10 | **Milestone 2: mid-project demo** (above). | | |
| 11 | Install CUDA-Q; QAOA on $n \leq 16$; test T6. | CUDA SA v2 (warp per chain); Nsight profiling. | Large-instance speed tests on all three platforms. |
| 12 | QAOA for $n$ up to 24; $\lambda$ study; (stretch) XY-mixer QAOA. | Tune kernels; profiling write-up. | Plots and tables; parallel-algorithm write-up. |
| 13 | **Milestone 3**: GitHub submission, demo, optional paper. | | |

Table 7: Schedule by academic week and member.

**Dependencies.** Member 1's Week 7 problem files feed everything. The scalar SA (Week 8) must exist before the AVX2 and CUDA versions can be checked against it (Week 9). QAOA (Weeks 11–12) needs only the Ising file and the exact-search answers.

# 8 Scope and Limitations

## 8.1 D1: Where the candidate drugs come from; Executed: B

| Option | Doability | Novelty |
|---|---|---|
| A. Sister project's ranked output | Depends on the other team (their list is due in Week 10). | Highest: a full screen-then-combine pipeline. |
| B. Curated list (pipeline reviews + approved AD drugs + published AD network hits) | High: tens of drugs; their closeness scores need only a small script. The data of [5] is ready to use. | Moderate: AD-specific and easy to explain, but small. |
| C. Both, through one shared table format | High: never blocked. | Like A when A arrives; like B otherwise. |

**Decision D1 (Executed: B)**

We built and tested everything on the curated list. We score the scaling pool with our own script and label those instances as speed tests, not biological results.

The curated list is small, which is good for biology and for exact answers, but too small to show interesting speedups. So the project uses two families of problems: small biological ones (about 12–24 drugs) for correctness and QAOA, and large ones (hundreds to a couple of thousand drugs) for speed tests.

### 8.2 D2: How to enforce "exactly $k$ drugs"

A commonly refenced textbook trick is to add a penalty $A(\sum_i x_i - k)^2$ to the energy. It works, but it links every pair of drugs, it needs a good value of $A$, and it makes almost every yes/no pattern invalid: for $n = 20$, $k = 3$, only 1,140 of about a million patterns (0.1%) have exactly three drugs [ESTABLISHED] (arithmetic).

**Decision Executed as-is**

- **Classical solvers** only ever look at valid combinations: exact search lists only $k$-drug combinations, and SA only swaps drugs in and out, so the count stays at $k$. No penalty is needed, and every answer is valid [PROPOSED].
- **QAOA** uses the penalty version, because standard QAOA cannot enforce a count by itself.

### 8.3 D3: Which search algorithm to parallelize

**Decision D3 (Executed: simulated annealing with swap moves)**

It is short to write, easy to explain, and has a clear vectorizable step. Running many independent chains is natural on a GPU. Other methods (tabu search, parallel tempering) were considered and left out.

### 8.4 D4: SIMD style

**Decision D4 (Executed: AVX2 assembly)**

SIMD has many variations of instruction sets such as AVX/AVX2/AVX-512 assembly, and the whole team has written `xmm`/`ymm` assembly before. The inner loop is written in NASM and called from C++. A C++ intrinsics version of the same loop is kept as a cross-check. AVX-512 is not used because many consumer CPUs lack it. For fairness, the sequential baseline is compiled with automatic vectorization turned off, and an automatically vectorized build is also reported.

### 8.5 D5: Making the three versions give identical answers

With floating-point numbers, the CPU and GPU round slightly differently, so after a few thousand steps their searches drift apart, and it becomes hard to tell a bug from rounding.

**Decision D5 (Executed: integer "deterministic mode")**

Scale all rewards and penalties to 32-bit integers, draw all random numbers from Philox with the same seed, and decide whether to accept a step with a shared lookup table. Then the scalar, AVX2 and CUDA versions must make exactly the same decisions, and comparing them catches bugs immediately. This is mainly a *debugging tool*: it makes the GPU version much easier to get right [PROPOSED].

### 8.6 D6: Quantum method

**Decision D6 (Incomplete: standard QAOA with the penalty)**

QAOA is the standard gate-based quantum method for this kind of problem and runs on CUDA-Q's GPU simulator. A variant that never produces invalid combinations (an "XY mixer" starting from a "Dicke state" [21, 22, 23]) would likely work better but needs a much harder circuit; it is a **stretch goal**, attempted only if standard QAOA is finished by Week 12 (**??**).

### 8.7 D7: Who does what

| Member | Responsibility |
|---|---|
| Member 1 (lead) | Data preparation; separation table; QUBO and Ising files and their tests; QAOA in CUDA-Q; C++ exact search |
| Member 2 | CUDA: exact-search kernel; SA kernels v1 (thread per chain) and v2 (warp per chain); the scalar C++ SA (the reference everyone checks against) |
| Member 3 | AVX2: assembly update loop and intrinsics twin; timing harness; benchmark scripts; plots and tables. Support Mmeber 2 |

Table 8: Roles. Member 1 only need to know QUBO or quantum computing: their solvers read only $w$, $P$ and $k$. Everyone reviews everyone's code, and Members 2 and 3 pair once on the CUDA SA kernel so knowledge is not held by one person.

### 8.8 Priorities

| Priority | Features |
|---|---|
| **MUST** | [DONE] Frozen data and candidate table; separation table; QUBO/Ising files with tests; exact search on CPU and GPU; SA in scalar C++, AVX2 assembly; timing tables. |
| **SHOULD** | [TODO] CUDA (v1) with deterministic mode; CUDA SA v2 (warp per chain); large-instance speed tests; standard QAOA on small instances; effect of $\lambda$ on the chosen drugs. |
| **OPTIONAL** | XY-mixer QAOA with Dicke state (stretch goal). |
| **OUT** | Everything in **??**: safety and side-effect data, splitting large problems for QAOA, parallel tempering, multi-core and other extra experiments, real quantum hardware. |

Table 9: Feature priorities.

### 8.9 Novelty Arguement

The individual methods are established: complementary exposure [5], simulated annealing [17], QAOA [8] and GPU state-vector simulation [9]. What is new is putting them together and measuring them carefully: (1) an AD-specific combination QUBO with a proven penalty setting and exhaustive tests; (2) one combination-search algorithm on scalar, SIMD and GPU hardware with identical decisions, so speed differences come from the hardware alone; and (3) to our knowledge, the first QAOA study of AD drug combinations, measured against exact answers. For a student research conference, (2) and (3) are the main claims.

# 9 Mid-Project Milestone Update

**Milestone 2 (Week 10) deliverable.** We present a live demo on the curated AD drugs attached in Github that shows:

1. the checked scoring function: on a 16-drug instance, an exhaustive Python check over all $2^{16}$ yes/no patterns confirms that the QUBO's best pattern is a valid $k$-drug combination and equals the exact-search answer;
2. CPU and GPU exact search returning the same best combination and the same tie count;
3. SA in scalar C++, AVX2 assembly;
4. a first sequential-vs-parallel timing table for exact search and SA;
5. the first best AD combinations for $k \in \{2, 3\}$ at two values of $\lambda$.

**Schedule.** Table 10 splits the work by member (roles in Table 8). Week 7 includes a deliberate CUDA ramp-up for Member 2 (and Member 3, who reviews the CUDA code).

| Week | Member 1 (lead) | Member 2 (CUDA) | Member 3 (AVX2, benchmarks) |
|---|---|---|---|
| 6 | Proposal (Milestone 1). Download and freeze data; fix table format. | Install CUDA toolkit; run the samples on the GTX 1660. | Set up repository, CMake, NASM build; Philox header. |
| 7 | Candidate table, closeness scores, separation table; QUBO/Ising builder; Python exhaustive checker. | **CUDA ramp-up**: vector add; a min-reduction kernel; a kernel that gives each thread its own Philox stream. | C++ exact search; reference energy function ("the judge"); timing harness. |
| 8 | Scalar C++ SA with integer deterministic mode (the reference). | GPU exact-search kernel; compare with Member 3's C++ version. | AVX2 assembly update loop and intrinsics twin; unit-test against scalar loop. |
| 9 | Formulation tests T1–T5 (**??**); first AD results. | CUDA SA v1 (thread per chain); cross-platform test T7. | AVX2 SA (scalar loop + assembly update); test T7; first timing table. |
| 10 | **Milestone 2: mid-project demo** (above). | | |
| 11 | Install CUDA-Q; QAOA on $n \leq 16$; test T6. | CUDA SA v2 (warp per chain); Nsight profiling. | Large-instance speed tests on all three platforms. |
| 12 | QAOA for $n$ up to 24; $\lambda$ study; (stretch) XY-mixer QAOA. | Tune kernels; profiling write-up. | Plots and tables; parallel-algorithm write-up. |
| 13 | **Milestone 3**: GitHub submission, demo, optional paper. | | |

Table 10: Schedule by academic week and member.

**Dependencies.** Member 1's Week 7 problem files feed everything. The scalar SA (Week 8) must exist before the AVX2 and CUDA versions can be checked against it (Week 9). QAOA (Weeks 11–12) needs only the Ising file and the exact-search answers.

## References


[1] J. L. Cummings *et al.*, "Alzheimer's disease drug development pipeline: 2026," *Alzheimer's & Dementia: Translational Research & Clinical Interventions*, 2026. Published May 5, 2026. TODO: volume, article number, full author list.

[2] A.-L. Barabási, N. Gulbahce, and J. Loscalzo, "Network medicine: a network-based approach to human disease," *Nature Reviews Genetics*, vol. 12, no. 1, pp. 56–68, 2011.

[3] J. Menche, A. Sharma, M. Kitsak, S. D. Ghiassian, M. Vidal, J. Loscalzo, and A.-L. Barabási, "Uncovering disease-disease relationships through the incomplete interactome," *Science*, vol. 347, no. 6224, p. 1257601, 2015.

[4] E. Guney, J. Menche, M. Vidal, and A.-L. Barabási, "Network-based in silico drug efficacy screening," *Nature Communications*, vol. 7, p. 10331, 2016.

[5] F. Cheng, I. A. Kovács, and A.-L. Barabási, "Network-based prediction of drug combinations," *Nature Communications*, vol. 10, p. 1197, 2019.

[6] F. Glover, G. Kochenberger, and Y. Du, "Quantum bridge analytics I: a tutorial on formulating and using QUBO models," *4OR*, vol. 17, pp. 335–371, 2019.

[7] A. Lucas, "Ising formulations of many NP problems," *Frontiers in Physics*, vol. 2, p. 5, 2014.

[8] E. Farhi, J. Goldstone, and S. Gutmann, "A quantum approximate optimization algorithm." arXiv:1411.4028, 2014.

[9] H. Bayraktar, A. Charara, D. Clark, S. Cohen, T. Costa, *et al.*, "cuQuantum SDK: A high-performance library for accelerating quantum science," in *Proceedings of the IEEE International Conference on Quantum Computing and Engineering (QCE)*, 2023. arXiv:2308.01999.

[10] D. Ramos, B. Coutinho, and D. Magano, "Network-based prediction of drug combinations with quantum annealing." arXiv:2512.20199, 2025. Code: `https://github.com/dmrapk/Drug-Combinations-using-Quantum-Annealers`. The sister specification cites a 2026 journal version in Quantum Machine Intelligence; TODO: confirm and update.

[11] F. Cheng, R. J. Desai, D. E. Handy, R. Wang, S. Schneeweiss, A.-L. Barabási, and J. Loscalzo, "Network-based approach to prediction and population-based validation of in silico drug repurposing," *Nature Communications*, vol. 9, p. 2691, 2018.

[12] J. Fang, P. Zhang, Y. Zhou, C.-W. Chiang, J. Tan, Y. Hou, *et al.*, "Endophenotype-based in silico network medicine discovery combined with insurance record data mining identifies sildenafil as a candidate drug for Alzheimer's disease," *Nature Aging*, vol. 1, pp. 1175–1188, 2021.

[13] TODO: authors, "Network proximity-based computational pipeline identifies drug candidates for different pathological stages of Alzheimer's disease," *Computational and Structural Biotechnology Journal*, 2023.

[14] TODO: authors, "Prioritizing repurposable drugs for Alzheimer's disease using network-based analysis with concurrent assessment of Long QT syndrome risk." `https://pmc.ncbi.nlm.nih.gov/articles/PMC12337645/`, 2025. PMID: 40791766.

[15] C. Bellenguez, F. Küçükali, I. E. Jansen, L. Kleineidam, S. Moreno-Grau, N. Amin, *et al.*, "New insights into the genetic etiology of Alzheimer's disease and related dementias," *Nature Genetics*, vol. 54, pp. 412–436, 2022.

[16] J. L. Cummings, Y. Zhou, G. Lee, *et al.*, "Alzheimer's disease drug development pipeline: 2025," *Alzheimer's & Dementia: Translational Research & Clinical Interventions*, vol. 11, no. 2, p. e70098, 2025.

[17] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi, "Optimization by simulated annealing," *Science*, vol. 220, no. 4598, pp. 671–680, 1983.

[18] D. J. Earl and M. W. Deem, "Parallel tempering: Theory, applications, and new perspectives," *Physical Chemistry Chemical Physics*, vol. 7, pp. 3910–3916, 2005.

[19] S. V. Isakov, I. N. Zintchenko, T. F. Rønnow, and M. Troyer, "Optimised simulated annealing for Ising spin glasses," *Computer Physics Communications*, vol. 192, pp. 265–271, 2015.

[20] J. K. Salmon, M. A. Moraes, R. O. Dror, and D. E. Shaw, "Parallel random numbers: as easy as 1, 2, 3," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC '11)*, 2011.

[21] S. Hadfield, Z. Wang, B. O'Gorman, E. G. Rieffel, D. Venturelli, and R. Biswas, "From the quantum approximate optimization algorithm to a quantum alternating operator ansatz," *Algorithms*, vol. 12, no. 2, p. 34, 2019.

[22] Z. Wang, N. C. Rubin, J. M. Dominy, and E. G. Rieffel, "XY mixers: Analytical and numerical results for the quantum alternating operator ansatz," *Physical Review A*, vol. 101, p. 012320, 2020.

[23] A. Bärtschi and S. Eidenbenz, "Deterministic preparation of Dicke states," in *Fundamentals of Computation Theory (FCT 2019)*, vol. 11651 of *Lecture Notes in Computer Science*, pp. 126–139, 2019.

[24] J. Cook, S. Eidenbenz, and A. Bärtschi, "The quantum alternating operator ansatz on maximum k-vertex cover," in *IEEE International Conference on Quantum Computing and Engineering (QCE)*, pp. 83–92, 2020. arXiv:1910.13483.

[25] L. Zhou, S.-T. Wang, S. Choi, H. Pichler, and M. D. Lukin, "Quantum approximate optimization algorithm: Performance, mechanism, and implementation on near-term devices," *Physical Review X*, vol. 10, p. 021067, 2020.

[26] NVIDIA Corporation, "NVIDIA CUDA-Q documentation: installation, single-GPU state-vector simulators, and Python API." `https://nvidia.github.io/cuda-quantum/latest/`, 2026. Accessed October 2026.

[27] NVIDIA Corporation, "cuQuantum documentation: Getting started and dependencies." `https://docs.nvidia.com/cuda/cuquantum/`, 2026. Accessed October 2026.

[28] R. Oughtred, J. Rust, C. Chang, B.-J. Breitkreutz, C. Stark, A. Willems, *et al.*, "The BioGRID database: A comprehensive biomedical resource of curated protein, genetic, and chemical interactions," *Protein Science*, vol. 30, no. 1, pp. 187–200, 2021.

[29] B. Zdrazil, E. Felix, F. Hunter, E. J. Manners, J. Blackshaw, S. Corbett, *et al.*, "The ChEMBL database in 2023: a drug discovery platform spanning multiple bioactivity data types and time periods," *Nucleic Acids Research*, vol. 52, no. D1, pp. D1180–D1192, 2024.

[30] S. L. Freshour, S. Kiwala, K. C. Cotto, A. C. Coffman, J. F. McMichael, J. J. Song, *et al.*, "Integration of the drug–gene interaction database (DGIdb 4.0) with open crowdsource efforts," *Nucleic Acids Research*, vol. 49, no. D1, pp. D1144–D1151, 2021.

| Component | Specification |
|---|---|
| Parallel paradigms | SIMD: AVX2 (256-bit `ymm` registers) in hand-written x86-64 assembly (NASM) for the inner loop, called from C++; the team already has x86 and `xmm`/`ymm` experience. A C++ intrinsics version of the same loop is kept as a cross-check. SIMT: CUDA C++. |
| GPU | NVIDIA GeForce RTX 5070 Ti: Blackwell (GB203), compute capability 12.0, 70 SMs with 8,960 CUDA cores, 16 GB GDDR7 on a 256-bit bus, 896 GB/s, 48 MB L2 cache [**?**]. GeForce cards are slow at 64-bit floating point, so all GPU code uses 32-bit integers (deterministic mode) or 32-bit floats. |
| CPU (baselines) | Intel Core Ultra 7 265KF (Arrow Lake): 8 performance cores and 12 efficiency cores, 20 threads (no hyper-threading), up to 5.5 GHz boost, 30 MB L3 and 36 MB L2 in total [**?**]; [RAM: size, DDR5 speed]. It supports AVX2 and FMA; AVX-512 is not available and not assumed. Same machine as the GPU. Timing runs are pinned to a performance core, and the clock speed is recorded, so that the efficiency cores do not distort the baselines. |
| OS | Linux (Ubuntu 22.04/24.04). Windows with WSL2 is acceptable for the C++/CUDA core; CUDA-Q is distributed for Linux, so QAOA runs either natively on Linux or under WSL2. GPU visibility under WSL2 is checked with `nvidia-smi` and a CUDA-Q test run before any experiments. |
| Toolchain | CUDA Toolkit 12.8 or newer (the first release that can compile for compute capability 12.0, `sm_120`) with an NVIDIA driver from the 570 series or newer, GCC/Clang with C++17, NASM, CMake; the Philox random-number generator [20] written once in a header that both CPU and GPU code include. |
| Quantum stack | CUDA-Q (Python, CUDA 12 build) with the `nvidia` target, which simulates on a single GPU through cuQuantum's cuStateVec [26, 9]. cuStateVec supports the Blackwell GPU family from version 1.8 (cuQuantum 25.03) [27], so we use a recent CUDA-Q release and test it on this card first. If installation fails: CUDA-Q's CPU simulator for small $n$, or cuQuantum's Python interface directly. |
| Analysis | Python 3.11+ (NumPy, SciPy, pandas, NetworkX for small reference checks, matplotlib). |
| Profiling | Nsight Systems and Nsight Compute (recent versions with Blackwell support) on the GPU; `perf` and Intel VTune on the CPU, with hardware counters read on native Linux where available. |
| Repository | GitHub: source, build and benchmark scripts, frozen data manifests (not the licensed raw data), the parallel-algorithm discussion, and the sequential-vs-parallel timing results required for Milestone 3. |

Table 4: Target platform.